

A GPU-based parallel Object kinetic Monte Carlo algorithm for the evolution of defects in irradiated materials

F. Jiménez, C.J. Ortiz *

Laboratorio Nacional de Fusión por Confinamiento Magnético – CIEMAT, Avenida Complutense 40, E-28040 Madrid, Spain



ARTICLE INFO

Article history:

Received 24 June 2015

Received in revised form 2 October 2015

Accepted 8 November 2015

Available online 17 December 2015

Keywords:

Kinetic Monte Carlo

GPU

Defect evolution

Parallel computing

ABSTRACT

In this work we present a parallel Object kinetic Monte Carlo (OkMC) computational model implemented using GPUs (graphics processing units) computing to simulate the evolution of interacting random walkers in materials. Different test simulations were performed under different conditions and compared to analytical solutions and sequential OkMC codes. The simulation results obtained with our algorithm show to be in excellent agreement with sequential OkMC codes and analytical solutions. The speedups reached with the GPU programming with respect to sequential OkMC codes in the test cases used here were in the range [30–100]. This increased efficiency allows to follow the evolution of millions of interacting particles in a relatively short computational time in contrast to classical OkMC codes. We expect this parallel kMC algorithm based on GPU programming to allow for investigations on defect evolution in materials using simulation boxes of realistic dimensions and physical times close to those achieved experimentally.

© 2015 EURATOM. Published by Elsevier B.V. All rights reserved.

1. Introduction

Many properties of solids, such as hardness [1], ductile-to-brittle transition temperature [2], conductivity [3,4] or tensile strength [5] are related to the presence of defects or impurities, in particular when there are out-of-equilibrium conditions. This occurs for instance when impurities are directly implanted such as in the semiconductor industry for the fabrication of transistors, or when the material is subject to neutron irradiation such as in fission reactors or in the future fusion reactors.

Therefore, simulating the spatial and temporal evolution of defects and impurities in materials irradiated or under irradiation conditions is essential to understand and predict their macroscopic properties and behaviour in non-equilibrium conditions. To do so, different models or numerical approaches can be used like Molecular Dynamics (MD) [6–11], kinetic Monte Carlo (kMC) [12–19] or mean field rate theory [15,20–24] (also sometimes called Cluster Dynamics). Each of these methods has advantages and drawbacks and allows to address different scales of time and space. In this paper we focus on kMC models.

In particular, in this work we shall focus on a subset of kMC denominated Object kMC (OkMC), which only follows the evolution of off-lattice defects, such as impurities, interstitials or

vacancies or their clusters thereof. This method has been widely used to simulate the evolution of defects or impurities in different materials and fields of research [25,15,26–30]. In this simulation framework, each object (defect or impurity) possesses a set of characteristics such as type, shape, migration energy or binding energy to other defects, which determine the rate of the events it can undergo [31,12,13,15,27]. It must be noted that the OkMC cannot predict the rate of events, they must be given a priori [25]. In this respect, the rate of events is mainly determined by the migration and binding energies of defects and can usually be accurately calculated using First-Principle calculations [32–35] and/or MD [36–39]. Since OkMC keeps the coordinates of each defect, it allows to account for spatial correlations between individual defects in the system [27], which is an advantage over other models such as the Rate Theory [15,40], which are based on a mean-field approximation. Moreover, since only off-lattice defects are followed, in contrast to MD that simulates the motion of every atom in the simulation box, OkMC models are not limited to short times [41] like MD but can reach physical times closer to those achieved experimentally. This makes OkMC a valuable tool for the investigation of defect kinetics in irradiated materials in quasi-realistic conditions.

OkMC models are based on the Bortz–Kalos–Leibowitz (BKL) algorithm [42] also known as *residence-time* algorithm. This algorithm is equivalent to the stochastic simulation algorithm (SSA) developed by Gillespie [43]. According to this algorithm, a list of

* Corresponding author.

E-mail address: christophe.ortiz@ciemat.es (C.J. Ortiz).

all possible event rates (the frequency line) is first formed. An event to carry out is selected randomly from the frequency line. Then, one of the particles that can undergo the selected event is chosen randomly and the event is carried out. Physical time is then advanced using a Poisson law to calculate the probable time interval that might have passed between two events. The procedure is then repeated until a specified physical time has been reached. Since only one event is carried out at each time step, OkMC models become computationally intensive [44] when the evolution of a large amount of defects must be simulated. For this reason, most OkMC codes using the standard BKL algorithm are usually limited to relatively small simulation boxes [45], in the order of $(200a_0)^3$, where a_0 is the lattice constant, in order to restrict the amount of particles to follow. Clearly, this strong restriction seriously hinders the study of defect evolution in realistic pieces of materials that can contain polycrystals and grain boundaries, which play an important role in the evolution of defects and/or impurities and thus, cannot be neglected [46–51]. Furthermore, small simulation boxes are inadequate to investigate the evolution of defects that form at low densities. This is the case for instance of dislocation loops (DLs) that form in irradiated Fe. For example, Arakawa et al. [52] observed that DLs form in Fe under electron irradiation at densities of about $10^{22} - 10^{23} \text{ m}^{-3}$. This means that with a typical OkMC simulation box with a volume of $(200a_0)^3$, it is only possible to follow the evolution of 1 to 20 DLs, which is clearly not enough statistical information for the prediction of defect evolution and their influence on a macroscopic level.

In order to address the evolution of defects in realistic conditions using the BKL algorithm, it seems thus unavoidable to increase the simulation box volume. This implies a significant increase of the number of particles to follow (at constant density), and, as a consequence, of the computational effort. Therefore, it is desirable to find some way to accelerate the BKL algorithm. For example, Martínez et al. [53] recently proposed a parallel version of the BKL algorithm using a multi-processor approach and the Message Passing Interface (MPI) paradigm. Though the authors observed a speedup of the algorithm, they observed that the achieved speedup tends to saturate as the number of processors increases, due to the increase of communication between processors.

In this work we present an alternative parallel BKL algorithm that we implemented using GPU (graphics processing unit) programming. Owing to their architecture, GPUs are capable of processing many data at the same time, so they are ideal for solving parallelisable problems [54]. For example, the NVIDIA® GeForce® GTX TITAN Black that we used in this work features 15 multiprocessors with 192 cores per multiprocessor, which yields a potential maximum of 240 concurrent blocks of 128 threads per block, i.e. 30,720 concurrent threads. As one can see, the parallelism that can be reached with a single graphics card is important, which explains why general-purpose programming on graphics processing units (GPGPUs) has risen in recent years [55] as a reliable and interesting alternative to inherently sequential CPU coding, especially in scientific computation. In addition, GPUs are known as energetically efficient devices as fewer watts per byte are needed than for CPUs.

The present article is organised as follows. In Section 2 we describe our parallel version of the BKL algorithm and highlight its advantages. In Section 3 different numerical tests are performed to validate our GPU-OkMC model. In particular, results are compared to those obtained with a CPU-based OkMC, as far as accuracy and performance are concerned. Finally, in order to demonstrate the efficiency of our GPU-OkMC, we reproduce a typical resistivity recovery experiment in Fe, which is a relevant problem in materials science [15,38,56–59].

2. Parallel OkMC algorithm

In this section we describe our parallel version of the BKL algorithm, which was implemented using the high parallelism offered by GPUs.

First, a particle array that resides in the global memory of the GPU is built and distributed in the simulation box. Then, the following operations are performed in parallel.

1. For each particle p_i able to undergo different events E_k with rate r_k , the sum of event rates is computed in parallel, as described by:

$$R_i = \sum_{k=1}^{q_i} r_k \quad (1)$$

with q_i being the total number of events that particle p_i can undergo.

2. The maximum sum of event rates $R_{\max}$ of all particles is determined in a parallel manner:

$$R_{\max} = \max\{R_i\} \quad (2)$$

3. Global time is advanced by the time step:

$$\delta t = \frac{\omega}{R_{\max}} \quad (3)$$

where $\omega > 0$ is a real number.

4. For each particle p_i , the number of times N_k that each event E_k with rate r_k will occur during the time step δt is determined using the Poisson distribution:

$$P(r_k, N_k; \delta t) = \frac{1}{N_k!} (r_k \delta t)^{N_k} \exp(-r_k \delta t) \quad (4)$$

5. Particles undergo the selected events in parallel, provided $N_k > 0$.
6. The surrounding of particles that have moved is analysed.
7. Particles that lie within a specified distance interact, provided interaction is allowed.

Here, we would like to emphasise the main differences between our parallel kMC algorithm and the original BKL [42] and the parallel algorithm proposed by Martínez [53]. On the one hand, in the standard BKL algorithm, only one event is performed during each time step. In the case of the parallel algorithm from Martínez et al. developed using MPI, at most K (number of processors used) events can occur during the same time step due to the subdivision of space into j subdomains. In our algorithm, space is not divided into subdomains. Instead, we consider that all particles are potentially able to undergo events during each time step, the probability of which is determined by the Poisson distribution (see Eq. (4)). In practice, this significantly increases the number of events that can occur in parallel during each time step. Therefore, the use of a GPU and the high parallelism it offers is necessary. Since the probability of each event (for each particle) explicitly depends on the time step (see Eq. (4)), it is first necessary to fix a value for the time step before events are carried out. In contrast, in the standard BKL or in the algorithm developed by Martínez et al., events are first carried out and then the probable time interval that might have passed between two events is calculated according to a Poisson law. Since each particle in our algorithm is considered, the time step that is selected must be independent of the total number of particles in the system. Furthermore, the time step cannot be arbitrary but must be limited by the highest event rate (step 2) in the system in order to preserve accuracy. Therefore, a natural choice is a time step that is inversely proportional to the highest sum of event rates in the system (Eq. (2)). On the one hand, the ω value

must be large enough so that a significant number of events can occur during each timestep. On the other hand, the selection of the ω value must be small enough so that the system as a whole does not change significantly and so that physical accuracy is preserved. This issue is similar to the one encountered in the tau-leaping method that was proposed by Gillespie [60] and will be addressed in Sections 2.1 and 3.3.

After the timestep has been selected, for each particle i the number of times N_k that an event E_k will occur is calculated using a Poisson law, as described by Eq. (4). The events are then selected randomly. For instance, if it is determined that a particle must perform 3 migrations and 1 dissociation during a time step, then migration and dissociation events are randomly selected until they are all performed. However, if a dissociation is selected before all migrations have been performed, then dissociation occurs and no more migrations are performed since the particle disappears in the dissociation process. Once all the selected events have been realised in parallel, the surrounding of each particle that has moved or dissociated must be analysed in order to determine the possible interaction partners. This corresponds to steps 6–7. As it is usually done in other OkMC models [31,12,27,15], in our model we assume that a reaction between two particles instantaneously takes place whenever two particles lie within a distance r_c , called the capture radius. This reflects the fact that the interaction range is limited and that the necessary time for a reaction to occur when species are close is much shorter than any diffusion process and can thus be seen as instantaneous. Following this assumption, only the close surrounding of those particles that have undergone an event is physically interesting and must be analysed. In practice, the distance between the particles that have moved and the other particles must be calculated to check whether they lie within the capture radius and could thus undergo a reaction. This step could be computationally costly if all the distances between neighbours had to be calculated at each time step. To avoid this, we used the collision detection algorithm that is described in chapter 32 of the book GPU Gems 3 [61]. According to this algorithm, protective spheres are drawn around each particle with radii equal to the largest capture radius of their defined interactions where they may be respectively involved. The cell where the particle centre is located is labelled ‘home cell’ while cells intersecting its protective sphere are called ‘phantom cells’. In order to avoid unnecessary searches for interaction partners through the whole system, space is divided into subdomains with a size that is at least larger than twice the largest capture radius r_c . This way, distances between particles are only calculated for particles belonging to the same subdomain (be it as ‘home’ or ‘phantom’ cells, for further reference see [61]), significantly reducing the computational effort. Owing to the division of space into subdomains, the inspection of the surrounding of each particle that has moved can clearly be achieved in parallel and independently in the GPU. In order to perform the calculation of distance between pairs of particles within a given subdomain and determine the possible interactions between particles, we used the GPU concept of thread blocks. The advantage of thread blocks is that the different threads they are composed of share the same block of shared memory, whose access is significantly faster than the one to the global memory. In our implementation, each thread block is assigned a subdomain and each thread of the block is in charge of the calculation of the distance between the particle that has moved and another particle within that subdomain. This way, the calculation of distances between possible interacting pairs is achieved in parallel, which significantly accelerates calculations. After the calculation of distances between potential interacting pairs have been achieved, it could happen that various particles lie within a distance lower than the capture radius. Thus, a criterion is necessary to decide which pairs should interact. At this point, several criteria can be considered. The most

intuitive one is that the closest pair is preferential over the others and should be the one to indeed react. Although this criterion seems logical, from the physical point of view, there is no evidence that reactions really take place this way and there are probably other factors to take into account, such as the energy landscape (shape of the interaction potential) between the different particles in presence. Another criterion is simply to consider, among all the particles within the capture radius, the first pair that appears in the list. This somehow takes into account the stochasticity of reactions and the fact that the energy landscape between particles cannot be known a priori. In this work, we chose to use a combination of both criteria. Particles in the same thread block are iterated through and allowed to react with the closest neighbour within the capture radius. The reaction thus takes place for this pair of particles, should they reside within the capture radius. This step is iteratively applied to the rest of particles in the thread block until all potential interacting pairs have reacted.

2.1. Selection of the time step

As we mentioned previously, in our algorithm the time step must be first fixed in order to calculate the probability that each particle has to undergo an event. Since each particle is considered separately from the others and at the same time, the time step must not depend on the number of particles in the system, in contrast to what is done in the standard BKL or in the Martínez algorithms. Indeed, in the original BKL algorithm, the probable time interval that might have passed between two events is inversely proportional to the sum of all event rates in the system. Thus, it depends somehow on the total number of particles in the system. If, for instance, we consider a simple system composed of N_{part} identical particles that can only perform one type of transition with event rate r , then the time step achieved in the standard BKL algorithm is simply given by:

$$\delta t_{\text{BKL}} = \frac{-\log(\zeta)}{N_{\text{part}} \cdot r} \quad (5)$$

where $\zeta \in (0, 1]$ is a uniform random variable. According to Eq. (5) we can see that the time step in the BKL algorithm gets smaller as the number of particles in the system increases. Furthermore, it can be proven that, if $X := -\log(\zeta)$, then X is distributed as an exponential distribution with mean value $\lambda = 1$. Provided the number of time steps is large enough in the simulation, the average time step in the BKL algorithm is thus:

$$\delta t_{\text{BKL}} \approx \frac{1}{N_{\text{part}} \cdot r} \quad (6)$$

In our algorithm, the time step value is related to the particle that displays the largest sum of event rates, according to Eqs. (2) and (3), and is thus independent of the number of particles in the system. Thus, if we consider the previous system of N_{part} identical particles, the time step achieved with our algorithm is simply $\delta t = 1/r$ and is in average N_{part} times larger than the one achieved with the standard BKL algorithm. In practice, this means that for typical systems containing several hundred thousands or millions of particles, with our algorithm the time step value is several orders of magnitude larger than that achieved with the standard BKL algorithm. This strongly reduces the number of computational steps that are necessary to reach a specified physical time. However, due to the large values that the time step can take in our algorithm, there exists a significant probability that a particle undergoes the same event several times during the same time step. This can be clearly seen in Fig. 1 where we plotted the probability (Eq. (4)) that an event E with rate r occurs N times during the same time step. Here we have assumed a time step $\delta t = 1/r$, i.e. $\omega = 1$ in Eq. (3).

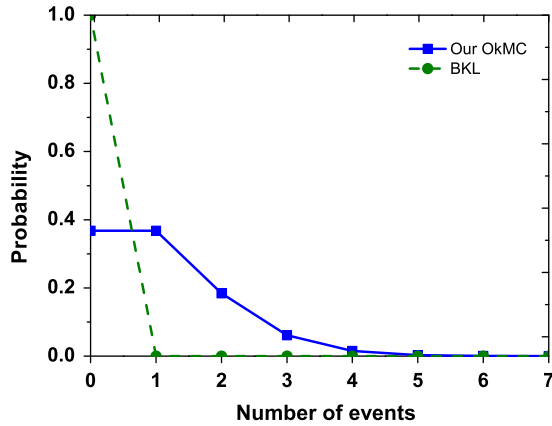


Fig. 1. Probability that a particle performs a given number of events during a time step according to our algorithm (squares) and to the BKL algorithm (circles).

For comparison, we also reported the probability of events for the standard BKL algorithm for the average time step given by Eq. (6), assuming a typical system containing $N_{\text{part}} = 200,000$ identical particles, which corresponds to a density $\rho = 6.84 \times 10^{19} \text{ cm}^{-3}$ in a $(500a_0)^3$ simulation box. As one can see, in the standard BKL algorithm, the probability that a particle undergoes no event at all during a time step is close to 1 whereas the probability that a particle undergoes one event is N_{part} times smaller, i.e. orders of magnitude smaller. Furthermore, the probability that a particle undergoes K times the same event during the same time step becomes negligible as it varies with $1/N_{\text{part}}^K$. This explains why in the BKL algorithm only one particle in the system experiences one event during a time step. In contrast, in our algorithm, the time step is such that the probability that an event occurs during a time step $\delta t = 1/r$ is relatively high (~ 0.37), as can be seen in Fig. 1. This implies that a large fraction of particles in the system can undergo an event during the same time step. The probability that the event occurs two and three times is also non-negligible as we can see in Fig. 1. However, since the surrounding of particles is only examined after the time step has passed, this might affect the accuracy of the solution. For instance, a large time step may allow a particle to undergo N migrations during the time step and to ignore its surroundings, and thus possible interactions, on $N_m - 1$ occasions. This might affect accuracy at large concentrations for which the distance between particles can be relatively small. In order to achieve a physically realistic approximation, this occurrence ought to be minimised. As mentioned previously, this problem is also encountered in the tau-leaping method [60] developed by Gillespie. By simply choosing small values of ω (Eq. (3)), i.e. small values of the time step, it is in principle possible to minimise the effects of this approximation, as can be seen in Fig. 2. In this figure we plotted the probability (Eq. (4)) that an event occurs several times during the same time step for different values of ω . As it is evidenced, this probability significantly decreases as the ω value does. For instance, for $\omega = 0.1$, the probability that an event occurs twice during the time step is already negligible. On the other hand, it is desirable to find a compromise and select a time step that is sufficiently large in order to keep a substantial simulation performance. However, it must be noted that in the different cases that we studied here, we did not observe a significant influence of ω on the result accuracy. Therefore, in the rest of the document, all simulations were performed taking $\omega = 1$, i.e. $\delta t = 1/R_{\text{max}}$. Furthermore, it seems natural to choose this value for ω since, according to the properties of the Poisson distribution, it corresponds to one event on average per time step for the particles that exhibit the highest sum of event rates.

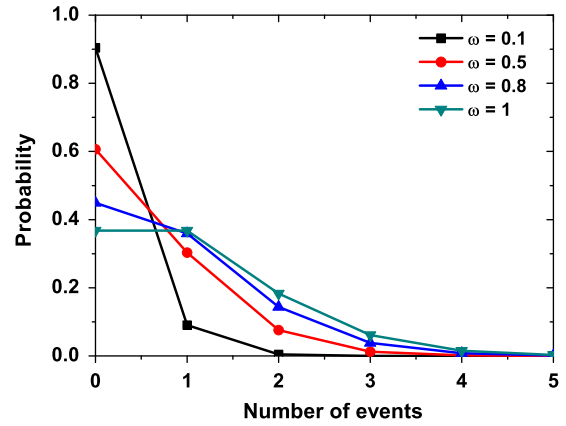


Fig. 2. Probability that a particle performs a given number of events during the time step δt for different values of ω .

3. Test cases

In this section, we show some test results simulated using our parallel OkMC algorithm. The OkMC model that we described in previous section was implemented using the programming framework CUDA 6.5 (Compute Unified Device Architecture). CUDA is an extension of C/C++ that was introduced by NVIDIA® in 2007 for graphics cards. The simulations shown here were performed on a NVIDIA® GeForce® GTX TITAN Black card with 6 GB of global memory and 2880 CUDA cores. As practical cases, tests were performed considering the evolution of self-interstitial atoms (SIA) and vacancies (V) in Fe.

3.1. Random walks of non-interacting particles

Since the evolution of defects in materials often occurs by means of random walks, here we simulated the diffusion of a large amount of non-interacting particles. In this simple case, results can be compared with analytical solutions. We simulated the random walk of five million SIAs in a $(36a_0)^2 \times 216a_0$ ($a_0 = 2.86 \text{ Å}$) box and initially located in a plane at $z = \frac{z_{\text{max}}}{2}$, with their x, y positions $0 \leq x \leq x_{\text{max}}$ and $0 \leq y \leq y_{\text{max}}$ being initially random. By letting these particles perform a one-dimensional random walk in the z direction, after a time t we expect their profile in said direction to describe a Gaussian function in accordance to Fick's laws of diffusion, provided the number of jumps is sufficiently high. This test was carried out assuming an attempt frequency $\nu_0 = 10^{13} \text{ s}^{-1}$ and a migration energy $E_m = 0.34 \text{ eV}$, which corresponds to the parameters of the SIA in Fe [32]. According to these parameters, the rate of the unique event (jump) that can occur in the system is:

$$r = \nu_0 \cdot \exp\left(-\frac{E_m}{k_B T}\right) \quad (7)$$

where k_B is the Boltzmann constant and T the temperature in K.

Simulations were performed for temperatures of $T = 130, 135$ and 140 K and a time $t_{\text{max}} = 30 \text{ s}$. Our results are reported in Fig. 3 and compared to a Gaussian function that represents the probability to find a particle between z and $z + \delta z$ after a time t [62]:

$$\text{Prob}(z, t) \cdot \delta z = \frac{1}{2(\pi D t)^{\frac{1}{2}}} \cdot \exp\left(-\frac{z^2}{4 D t}\right) \cdot \delta z \quad (8)$$

where D is the macroscopic diffusion coefficient of SIAs that is defined using the attempt frequency ν_0 and the migration energy E_m as follows:

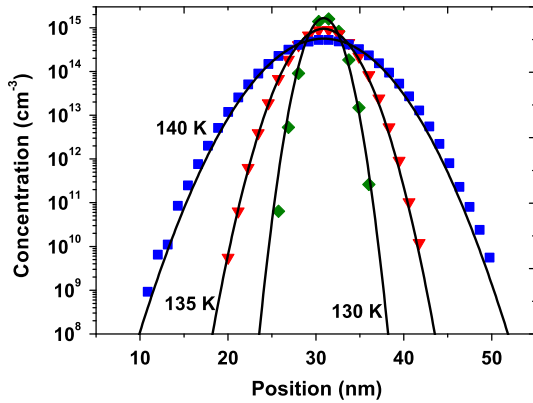


Fig. 3. Comparison between the 1D profiles resulting from the diffusion of five million particles during 30 s for three different temperatures as simulated by our GPU-based OKMC code (symbols) and the analytical solution (lines).

$$D = \frac{v_0 \lambda^2}{2} \exp\left(-\frac{E_m}{k_B T}\right) \quad (9)$$

where λ is the jump length. The factor 1/2—instead of the typical 1/6—corresponds to random walks in one dimension, as it is assumed here. As one can see in Fig. 3, results obtained with our GPU-OkMC are in very good agreement with the analytical solution.

In order to estimate the performance achieved with our GPU-OkMC we simulated a similar case that was considered by Martinez et al. [53], the migration of random walkers with no interaction. The authors simulated the migration of 2^{22} random walkers (~ 4 million particles) using their parallelised OkMC. According to their results, they achieved a speedup of about 30 using 64 processors, in comparison to the standard BKL algorithm used with a single processor. Here we simulated the migration of four million SIAs at a temperature of 135 K for 10 s. Simulations performed with the BKL algorithm were achieved using the sequential OkMC code LAKIMOCA [31] version o128p. LAKIMOCA simulations were performed on a CPU AMD Opteron 6174@2.2 GHz. A runtime of 563 s was necessary using the BKL algorithm and LAKIMOCA code. Only 1.9 s were necessary with our algorithm. This corresponds to a speedup of about 296.

3.2. Diffusion of non-interacting particles in the presence of absorbing surfaces

The next test case involves the random walk of non-interacting particles in the presence of absorbing surfaces. This case is of high relevance in irradiated materials as point defects that are created by displacement cascades, such as interstitials and vacancies, tend to recombine when they reach the surface.

In this case, we simulated the evolution of five million SIAs initially distributed randomly in a volume of $(400a_0)^3$, which corresponds to a density $\rho = 3.34 \times 10^{21} \text{ cm}^{-3}$. Particles were allowed to perform random walks in the three dimensions during 200 s for different temperatures. Periodic boundary conditions were assumed in x and y directions whereas absorbing boundary conditions were assumed at $z = 0$ and $z = 400a_0$. The same jump frequency and the migration energy were used to calculate the event rate (jump) as in previous subsection. In order to validate our GPU-OkMC model, simulations with classical diffusion equations and Dirichlet boundary conditions were also performed. Fig. 4 shows the evolution of the total number of particles with respect to time for the different temperatures. For comparison, we also reported the results obtained with the classical diffusion equations. As we can see, the results obtained with our

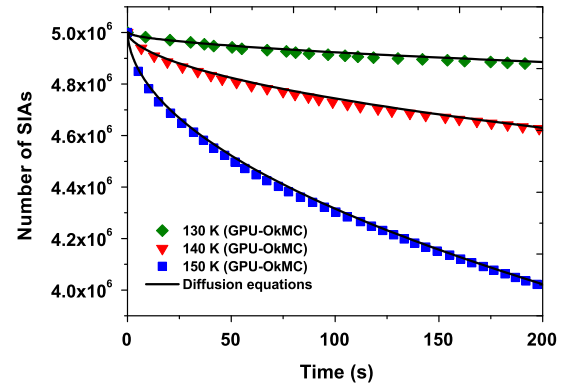


Fig. 4. Evolution of the total number of random walkers as a function of time in the presence of two absorbing surfaces. GPU-OkMC results (symbols) are compared to those obtained with diffusion equations (lines).

GPU-OkMC model are in excellent agreement with those obtained with diffusion equations. In the same way, Fig. 5 shows the depth profiles of particles for the different temperatures considered here. The very good agreement obtained with the diffusion equations clearly evidences that our GPU-OkMC accurately accounts for the recombination of particles on absorbing surfaces.

3.3. Recombination of interacting particles

In this subsection we address the case of interacting random walkers, which is of high interest in many fields of research. Indeed, the evolution of many systems observed in biology, chemistry or in irradiated materials is often determined by the kinetics of diffusion-limited reactions $A + B \rightarrow AB$. As explained in Section 2, it is assumed that particles A and B instantaneously react if the distance between both is smaller than r_c . As a practical case of interest, here we shall consider the case of the annihilation of SIAs and vacancies that form by collision cascades in irradiated Fe. This can be represented by the reaction $I + V \rightarrow 0$. The same physical parameters (attempt frequency and migration energy) as in previous subsections were used for SIAs to calculate the event rate corresponding to the jump frequency. To calculate the jump frequency of vacancies in Fe, we used a migration energy of 0.67 eV [32,15] and the same attempt jump frequency as for SIAs. Regarding the capture radius for the annihilation reaction, a value of $3.3a_0$ was used to describe the range of interaction between interstitials and vacancies, as it is commonly accepted in the community [32,15].

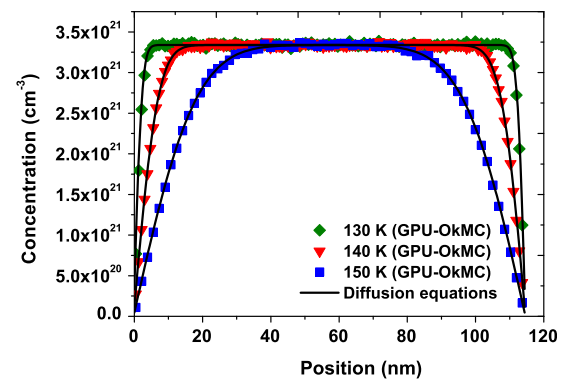


Fig. 5. Depth profiles of particles in z direction for three different temperatures. GPU-OkMC results are represented by symbols while those predicted by diffusion equations are represented by lines.

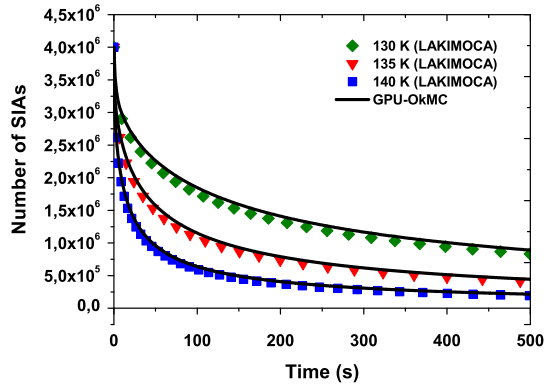


Fig. 6. Evolution of the number of SIAs as a function of time for different temperatures. Results are compared to those obtained by the sequential OkMC LAKIMOCA (courtesy of C. Domain).

We considered the diffusion and recombination of 4 million SIAs and 4 million vacancies initially randomly distributed in a hypothetical Fe material with volume of $(200a_0 \times 200a_0 \times 70000a_0)$, such that the initial Frenkel pair density is $\rho = 6.11 \times 10^{19} \text{ cm}^{-3}$. This corresponds to a material with a thickness of about $20 \mu\text{m}$, i.e. of realistic dimensions. The evolution of the system was simulated for temperatures of 130, 135 and 140 K for 500 s. Periodic boundary conditions in all directions were assumed. Thus, in the present case, particles can only disappear by recombination. For comparison of accuracy and performances, simulations were also performed using a sequential (CPU) OkMC code, i.e. based on the standard BKL algorithm. For this, LAKIMOCA [31] code version o128p was used. LAKIMOCA simulations were performed on a CPU AMD Opteron 6174@2.2 GHz. In Fig. 6, the evolution of the number of SIAs is shown as a function of time for the different temperatures considered here. The results obtained with the sequential OkMC LAKIMOCA are also reported for comparison. Clearly, the agreement between both models is excellent, which again validates our parallel algorithm and its implementation using GPU programming.

Table 1 summarises the runtime achieved to perform the simulations described above using our GPU-OkMC. Results were obtained for $\omega = 1$ (see Eq. (3)). For comparison, we also reported the runtimes achieved using the CPU-based OkMC LAKIMOCA code. As we can see, our algorithm shows a significant speedup in the range [30–40]. It is worth noting that these speedups were achieved using a commercial GPU NVIDIA® GeForce® GTX TITAN Black card with 6 GB of global memory and 2880 CUDA cores. It is expected that even larger speedups could be achieved by simply using a more efficient GPU, i.e. with more memory and more CUDA cores.

In order to examine the influence of the particle density and choice of time step width on accuracy and speedup, as discussed in Section 2.1, the case $T = 135 \text{ K}$ was repeated with different values of ω ranging from 0.1 to 5 and different initial densities ρ of SIA and V ranging from $6.11 \times 10^{17} \text{ cm}^{-3}$ to $6.11 \times 10^{20} \text{ cm}^{-3}$. In each case the initial number of particle was the same (4 million of each species) and the volume was varied in order to change the initial density. As previously, results were compared to the result obtained with the sequential OkMC code LAKIMOCA. As we can see in Fig. 7, only a slight deviation of the results is observed for the largest timestep ($\omega = 5$) and for medium and high densities. This is expected since for large values of ω , i.e. for large timestep widths, particles may perform several jumps during the same timestep. Thus, for medium or high concentrations interactions can be missed since the neighbourhood of particles is only examined at the end of each timestep. Nevertheless, in general, in the

Table 1

Simulation times achieved by our GPU-OkMC using a NVIDIA® GeForce® GTX TITAN Black card, compared to the runtimes needed by the sequential OkMC code LAKIMOCA using a CPU AMD Opteron 6174@2.2 GHz. Numbers in brackets indicate the corresponding speedup.

Simulation temp.	CPU time	GTX TITAN Black time
130 K	64 min	100 s (38.3×)
135 K	98 min	183 s (32.2×)
140 K	132 min	274 s (28.9×)

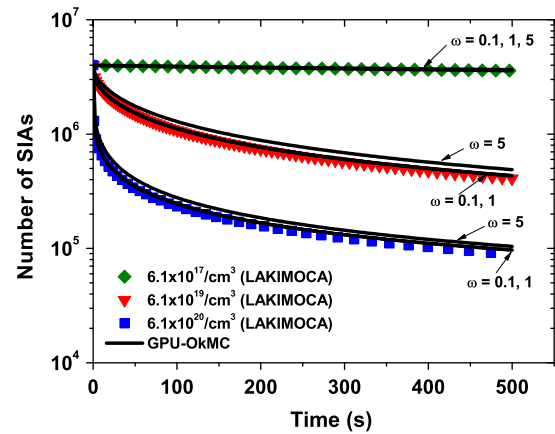


Fig. 7. Evolution of the number of SIAs at $T = 135 \text{ K}$ as a function of time for different initial densities and values of ω . Results are compared to those obtained by the sequential OkMC LAKIMOCA (courtesy of C. Domain).

cases studied here, it was easy to find a good compromise between performance and accuracy for values of $\omega > 1$.

The corresponding speedup achieved for the different values of the density and of ω is reported in Table 2. As we can see, speedup can be significantly improved by reasonably increasing the ω value, i.e. the time step, from 1 to 2. Indeed, larger time step values imply that less computational steps are necessary to reach a specified physical time.

Additionally, in order to distinguish between the gain achieved by the parallel algorithm proposed here from the one achieved by the implementation, we performed additional simulations where the standard BKL algorithm was used in the GPU. To do so, we used the definition of the time step as given in the BKL algorithm (see Eq. (5)). This leads to a much smaller time step and consequently, in average, only one event is carried out per time step, as it is expected in the BKL algorithm. As an example, we performed a simulation at 130 K for a time of 10 s in a volume of $(200a_0 \times 200a_0 \times 700a_0)$ with 40,000 SIAs and 40,000 Vs, which corresponds to a density of $6.11 \times 10^{20} \text{ cm}^{-3}$ for each species. In this condition, the time step predicted by the BKL is approximately equal to $3.79 \times 10^{-5} \text{ s}$. To achieve the simulation using the BKL algorithm in the GPU, a runtime of 1493 s was necessary. In contrast, when we use our algorithm, a time step of 1.51 s is obtained using Eq. (3) and $\omega = 1$. In this case, only 87 ms were necessary to

Table 2

Speedup achieved with our GPU-OkMC algorithm for different particle densities and values of ω in comparison to LAKIMOCA runtimes for the case $T = 135 \text{ K}$.

$\rho \text{ (cm}^{-3}\text{)} \omega$	0.1	0.5	1.0	2.0	5.0
6.1×10^{17}	12.9×	51.6×	88.4×	157.7×	375.6×
6.1×10^{19}	4.5×	17.8×	32.2×	51.1×	115.6×
6.1×10^{20}	3.3×	12.7×	20.5×	33.3×	62.9×

achieve the simulation. This evidences that when both algorithms are used in the GPU, an important speedup of about 17,100 is achieved between the BKL algorithm and the algorithm proposed in this work. Actually, this is not surprising. As we said in the introduction, the GPU used in this work is able to launch approximately 30,000 threads in parallel. This means that if our algorithm is used, approximately 30,000 particles can move simultaneously. However, if the BKL is used in the GPU, in average, only one particle move per time step, ie, about 30,000 times less than what can be done with our parallel algorithm. Hence, it is not surprising that we get a speedup in the order of 30,000 (exactly 17,100) between the BKL and our algorithm when they are both used in the GPU. The speedup obtained when comparing the performance of our algorithm with the serial code LAKIMOCA is of course not so high since the implementation of the BKL algorithm is likely optimised for a CPU use. Moreover, GPU processors are significantly slower than CPUs.

3.4. Application: Simulation of a resistivity recovery experiment

In order to demonstrate the efficiency of our GPU-OkMC to simulate the evolution of a large amount of interacting particles in realistic conditions, we simulated a typical resistivity recovery (RR) experiment in a Fe sample with realistic dimensions. This type of experiment is of importance as it allows to investigate the kinetics of defects in metals [63–65]. In a typical RR experiment, the sample is first cooled down to a very low temperature (4–77 K) and is then irradiated with electrons, such that only Frenkel pairs (I - V) are produced. At these temperatures, self-interstitials and vacancies are immobile and thus their recombination is inhibited. Temperature is then increased by δT and kept constant for a fixed amount of time, in general in the order of a few minutes. Resistivity ρ is measured at the end of the isothermal annealing and the process is repeated until a preset temperature is reached. Since resistivity reflects the total number of defects that affect electron diffusion in the metal, RR experiments are a valuable tool to study the evolution of defects and the thermally-activated mechanisms they undergo. In practice, a typical RR curve exhibits several drops, reflecting the activation of recombination processes. Indeed, as temperature increases, the migration of the different defects gets thermally activated, allowing them to diffuse and recombine with defects of the opposite type. As a result, if the derivative of the total number of defects is plotted with respect to temperature, a typical RR spectrum exhibits several peaks, also called recovery stages, corresponding to the different thermally-activated mechanisms [15,38,57] that take place in the metal.

To simulate a typical RR experiment in Fe, we considered the evolution of SIAs and Vs in pure Fe, assuming that they both migrate and can recombine, as described in previous subsection. The formation of SIA or V clusters or other mechanisms were neglected here. We emphasise that our goal is not to propose a physically-based model to explain the different features of the kinetics of defects in Fe but to show that, taking into account basic mechanisms and Fe parameters, our GPU-OkMC can be used to reproduce typical experiments using realistic simulation boxes representative of real samples with a large amount of particles and in short runtimes.

Here we consider a simulation box similar to that of a typical sample thickness in a RR experiment, with a thickness of about 20 μm ($70000a_0$). For the sake of simplicity, the effect of electron irradiation, i.e. the formation of Frenkel pairs during irradiation, was emulated by distributing randomly an equal number of interstitials and vacancies in the simulation box. In the present case we used 10 million SIAs and 10 million vs. The evolution of the system was then simulated for temperatures starting from 77 K up to 140 K with increase steps of $\frac{\delta T}{T} = 0.008$, and an annealing time

$t = 300$ s at each temperature. Each RR curve simulated here consists thus of 77 simulations at different temperatures. This was done for different values of x_{max} and y_{max} , in order to vary the volume of the simulation box and thus the initial concentration of particles. Values for the initial density of Frenkel pairs range from $6.11 \times 10^{18} \text{ cm}^{-3}$ to $6.11 \times 10^{20} \text{ cm}^{-3}$. Like in previous cases, $\omega = 1$ was used to compute time step values.

In Fig. 8, we show the derivative of the total number of defects as a function of temperature for different initial concentrations (different x_{max} and y_{max}). As the figure evidences, two different peaks (or stages) can be observed, depending on the initial concentration. When the initial concentration is sufficiently high, our GPU-OkMC predicts a first peak at about 107 K, which corresponds to the so-called I_D stage reported in literature [57,32] for Fe. This stage corresponds to the recombination of correlated interstitials and vacancies, i.e. that are spatially close. When the migration of SIAs is thermally activated, the probability that they recombine with their vacancy after the first jump is relatively high, giving rise to a recombination at relatively low temperature. The temperature at which our GPU-OkMC predicts this stage to appear is in perfect agreement with what is observed experimentally [57] and predicted theoretically [32]. Our GPU-OkMC also predicts a second peak at a temperature of about 120–130 K, in very good agreement to what is reported in the literature [57,32]. In contrast to previous stage (I_D), this stage, commonly denoted stage I_E , corresponds to the recombination of uncorrelated I - V pairs. It is related to the recombination of interstitials that must perform several jumps before they can find a vacancy and recombine. As one can see in Fig. 8, this peak shifts towards higher temperatures as initial concentration decreases. This is expected and experimentally observed [58] since for lower particle densities, the mean distance between particles increases and thus, interstitials need to perform more jumps to find a vacancy to recombine with. As a result, it becomes more difficult for interstitials to find a vacancy in a limited amount of time as the initial concentration decreases. Hence, higher temperatures are necessary for them to perform the required number of jumps before they can recombine with a vacancy, leading to the shift in temperature of stage I_E .

In Table 3 we reported the runtimes that were necessary to achieve the simulations shown in Fig. 8 for the different initial concentrations, the initial number of particles being 20 million in all cases. As one can see, our GPU-OkMC allows to simulate the evolution of a large number of particles and reproduce a realistic experiment in very reasonable runtimes. Where an OkMC based on the standard BKL algorithm would likely require days of calculations, our parallel OkMC algorithm implemented using

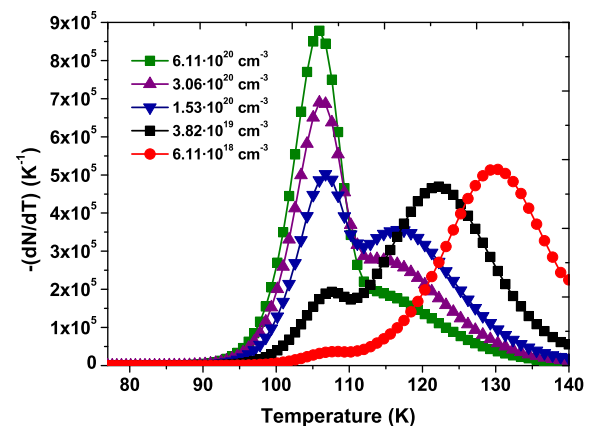


Fig. 8. Resistivity recovery spectra in Fe simulated with our GPU-OkMC and corresponding to different initial SIA and V densities.

Table 3

Size of simulation box, initial concentration of defects and respective runtimes corresponding to Fig. 8.

Simulation box	Initial density of defects (cm^{-3})	Runtime (s)
$100a_0 \times 100a_0 \times 70000a_0$	1.22×10^{21}	400.7
$100a_0 \times 200a_0 \times 70000a_0$	6.12×10^{20}	476.6
$200a_0 \times 200a_0 \times 70000a_0$	3.06×10^{20}	631.4
$400a_0 \times 400a_0 \times 70000a_0$	7.64×10^{19}	1270
$1000a_0 \times 1000a_0 \times 70000a_0$	1.22×10^{19}	3481

GPU programming only requires computational times between 6 min and slightly less than 1 h in the worst case (lowest concentration). Longer runtimes were indeed obtained in the cases where the system is diluted due to the fact that the probability of encounter/recombination between SIAs and Vs is lower and thus, a larger amount of particles is present in the system during the simulations.

4. Conclusions

In this work we have proposed a parallel Object kinetic Monte Carlo algorithm that was implemented using a GPU programming approach to simulate the evolution of large amounts of interacting particles in materials. In our algorithm, the time step value is first fixed and then the Poisson probability that an event occurs is computed independently and in parallel for each particle. This allows to increase the time step values by orders of magnitude in contrast to the standard BKL algorithm. At the same time, this allows a large fraction of particles to undergo events in parallel during the same time step, in contrast to the standard BKL algorithm that allows only one event to occur per time step. As a result, the number of computational steps to be performed to reach a specified physical time is significantly reduced.

Accuracy and performances of our GPU-OkMC code have been tested under different conditions. In the case of random walks under reflective or absorbing surfaces, the results obtained with our GPU-OkMC are in excellent agreement with those obtained with classical Fick's laws of diffusion. In the case of diffusion of interacting particles, the results obtained with our GPU-OkMC are in excellent agreement with those obtained with a OkMC code based on the standard BKL algorithm. Performances of our GPU-OkMC have been compared to those of a CPU-based OkMC, evidencing speedups in the range [30–40] with constant time step $\delta t = 1/R_{\max}$, i.e. for $\omega = 1$. Larger speedups were obtained by using larger values for ω at the expense of a slight loss of accuracy. It must be noted that even higher speedups could be obtained by using a more efficient graphics card than the one that was used in the present study.

In order to evidence the usefulness of our GPU-OkMC to simulate realistic experiments, we have reproduced a typical resistivity recovery (RR) experiment assuming an irradiated Fe sample of realistic sizes containing 20 million I - V pairs. By only taking into account the migration and recombination of interstitials and vacancies in Fe, our GPU-OkMC was able to accurately predict the main features of RR spectra in Fe that are observed experimentally and reported in the literature. On average, our GPU-OkMC was able to reproduce a complete RR experiment with runtimes between 6 min and less than 1 h, when several days of computation would have been necessary with a OkMC code based on the standard BKL algorithm.

We believe this parallel OkMC algorithm implemented using a GPU programming approach will open new possibilities in the investigation of defects in irradiated materials or in other fields

of research as it allows now to study the evolution of a large amount of interacting species in samples of realistic dimensions.

Acknowledgements

This work has been carried out within the framework of the EUROfusion Consortium and has received funding from the Euratom research and training programme 2014–2018 under grant agreement No. 633053. The views and opinions expressed herein do not necessarily reflect those of the European Commission. This work has been partially supported by Ministerio de Economía y Competitividad (MINECO) of Spain under ENE2012-39787-C06-01 project. The authors acknowledge M. Cárdenas from CIEMAT for helpful discussions related to GPU programming and N. Castin from SCK-CEN for valuable discussions on OkMC. The authors also acknowledge C. Domain from EDF for complementary simulations with LAKIMOCA code.

References

- [1] A.S.H. Hameed, S. Rohani, W.C. Yu, C.Y. Tai, C.W. Lan, J. Cryst. Growth 297 (2006) 146–151.
- [2] R. Schaeublin, J. Henry, Y. Dai, C.R. Phys. 9 (2008) 389–400.
- [3] F.J. Morin, J.P. Maita, Phys. Rev. 96 (1954) 28.
- [4] G.L. Pearson, J. Bardeen, Phys. Rev. Lett. 85 (1949) 865.
- [5] X. Wu, X. Pan, M. Li, J.F. Stubbins, J. Nucl. Mater. 343 (2005) 302–307.
- [6] R. Car, M. Parrinello, Phys. Rev. Lett. 55 (1985) 2471.
- [7] W.J. Phythian, R.E. Stoller, A.J.E. Foreman, A.F. Calder, D.J. Bacon, J. Nucl. Mater. 223 (1995) 245–261.
- [8] K. Nordlund, M. Ghaly, R.S. Averback, M.J. Caturla, T.D. de la Rubia, J. Tarus, Phys. Rev. B 57 (1998) 7556–7570.
- [9] A.F. Calder, D.J. Bacon, J. Nucl. Mater. 207 (1993) 25–45.
- [10] J.D. Bacon, T.D. de la Rubia, J. Nucl. Mater. 216 (1994) 275–290.
- [11] B. Sadigh, T.J. Lenosky, S.K. Theiss, M.J. Caturla, T.D. de la Rubia, M.A. Foad, Phys. Rev. Lett. 83 (1999) 4341–4344.
- [12] C. Becquart, C. Domain, U. Sarkar, A. DeBacker, M. Hou, J. Nucl. Mater. 403 (2010) 75–88.
- [13] C. Becquart, A. Barbu, J.L. Bocquet, M.J. Caturla, C. Domain, C.-C. Fu, S.I. Golubov, M. Hou, L. Malerba, C.J. Ortiz, A. Souidi, R.E. Stoller, J. Nucl. Mater. 406 (2010) 39–54.
- [14] F. Soisson, C. Becquart, N. Castin, C. Domain, L. Malerba, E. Vincent, J. Nucl. Mater. 406 (2010) 55–67.
- [15] C.J. Ortiz, M.J. Caturla, Phys. Rev. B 75 (2007) 184101.
- [16] E. Vincent, C.S. Becquart, C. Domain, J. Nucl. Mater. 351 (2006) 88–99.
- [17] E. Vincent, C.S. Becquart, C. Domain, Nucl. Instrum. Meth. B 255 (2007) 78–84.
- [18] D. Terentyev, G. Bonny, N. Castin, C. Domain, L. Malerba, P. Olsson, V. Molodtsov, R.C. Pasianot, J. Nucl. Mater. 409 (2011) 167–175.
- [19] I. Martín-Bragado, Nucl. Instrum. Meth. Phys. Res. B 303 (2013) 184–187.
- [20] C.J. Ortiz, P. Pichler, T. Fühner, F. Cristiano, B. Colombeau, N.E.B. Cowern, A. Claverie, J. Appl. Phys. 96 (2004) 4866–4877.
- [21] Y.G. Li, W.H. Zhou, R.H. Ning, L.F. Huang, Z. Zeng, X. Ju, Commun. Comput. Phys. 11 (2012) 1547–1568.
- [22] E. Meslin, A. Barbu, L. Boulanger, B. Radigue, P. Pareige, K. Arakawa, J. Nucl. Mater. 382 (2008) 190–196.
- [23] A. Barbu, E. Clouet, Solid State Phenom. 129 (2007) 51–58.
- [24] T. Jourdan, G. Bencteux, G. Adjanor, J. Nucl. Mater. 444 (2014) 298–313.
- [25] M.J. Caturla, N. Soneda, E. Alonso, B.D. Wirth, T.D. de la Rubia, J.M. Perlado, J. Nucl. Mater. 276 (2000) 13–21.
- [26] H. Xu, Y.N. Osetsky, R.E. Stoller, J. Nucl. Mater. 423 (2012) 102–109.
- [27] L. Malerba, C. Becquart, C. Domain, J. Nucl. Mater. 360 (2007) 159–169.
- [28] G.J. Galloway, G.J. Ackland, Phys. Rev. B 87 (2013) 104106.
- [29] M. Aboy, L. Pelaz, L.A. Marqués, P. López, J. Barbolla, R. Duffy, V.C. Venezia, P.B. Griffin, Appl. Phys. Lett. 86 (2005) 031908.
- [30] L. Pelaz, L.A. Marqués, M. Aboy, J. Barbolla, G.H. Gilmer, Appl. Phys. Lett. 82 (2003) 2038.
- [31] C. Domain, C.S. Becquart, L. Malerba, J. Nucl. Mater. 335 (2004) 121–145.
- [32] C.-C. Fu, J.D. Torre, F. Willaime, J.-L. Bocquet, A. Barbu, Nat. Mater. 4 (2005) 68–74.
- [33] C.-C. Fu, F. Willaime, P. Ordejón, Phys. Rev. Lett. 92 (2004) 175503.
- [34] C.-C. Fu, F. Willaime, J. Nucl. Mater. 367–370 (2007) 244–250.
- [35] L. Ventelon, F. Willaime, C.-C. Fu, M. Heran, I. Ginoux, J. Nucl. Mater. 425 (2012) 16–21.
- [36] E. Zarkadoulas, S.L. Daraszewicz, D.M. Duffy, M.A. Seaton, I.T. Todorov, K. Nordlund, M.T. Dove, K. Trachenko, J. Phys.: Condens. Matter 25 (2013) 125402.
- [37] C. Björkas, K. Nordlund, M.J. Caturla, Phys. Rev. B 85 (2012) 024105.
- [38] D. Terentyev, P. Olsson, T.P.C. Klaver, L. Malerba, Comput. Mater. Sci. 43 (2008) 1183–1192.

- [39] L. Malerba, G.J. Ackland, C.S. Becquart, G. Bonny, C. Domain, S.L. Dudarev, C.-C. Fu, D. Hepburn, M.C. Marinica, P. Olsson, R.C. Pasianot, J.M. Raulot, F. Soisson, D. Terentyev, E. Vincent, F. Willaime, J. Nucl. Mater. 406 (2010) 7–18.
- [40] R.E. Stoller, S.I. Golubov, C. Domain, C.S. Becquart, J. Nucl. Mater. 382 (2008) 77–90.
- [41] W. Cai, V.V. Bulatov, J.F. Justo, A.S. Argon, S. Yip, Comput. Mater. Sci. 23 (2002) 124–130.
- [42] A.B. Bortz, M.H. Kalos, J.L. Leibowitz, J. Comput. Phys. 17 (1975) 10–18.
- [43] D.T. Gillespie, J. Comput. Phys. 22 (1976) 403.
- [44] A.F. Voter, Introduction to the kinetic Monte Carlo method, in: K.E. Sickafus, E. A. Kotomin, B.P. Uberuaga (Eds.), *Radiation Effects in Solids*, Springer, Dordrecht, Netherlands, 2007.
- [45] J. Wei, F.E. Kruis, J. Comput. Phys. 249 (2013) 67–79.
- [46] M.A. Tschopp, K.N. Solanki, F. Gao, X. Sun, M.A. Khaleel, M.F. Horstemeyer, Phys. Rev. B 85 (2012) 064108.
- [47] X.-M. Bai, A.F. Voter, R.G. Hoagland, M. Nastasi, B.P. Uberuaga, Science 327 (2010) 1631–1634.
- [48] G.S. Was, J.P. Wharry, B. Frisbie, B.D. Wirth, D. Morgan, J.D. Tucker, T.R. Allen, J. Nucl. Mater. 411 (2011) 41–50.
- [49] T. Allen, J. Cole, J. Gan, G. Was, R. Dropek, E. Kenik, J. Nucl. Mater. 342 (2005) 90–100.
- [50] J. Busby, G. Was, E. Kenik, J. Nucl. Mater. 302 (2002) 20–40.
- [51] D. Terentyev, X. He, E. Zhurkin, A. Bakaev, J. Nucl. Mater. 408 (2011) 161–170.
- [52] K. Arakawa, H. Mori, K. Ono, J. Nucl. Mater. 307 (2002) 272–277.
- [53] E. Martínez, J. Marian, M.H. Kalos, M. Preciado, J. Comput. Phys. 227 (2008) 3804–3823.
- [54] C. Navarro, N. Hitschfeld-Kahler, L. Mateu, Commun. Comput. Phys. 15 (2014) 285–329.
- [55] R. Farber, *CUDA Application Design and Development*, Morgan Kaufman, Waltham, USA, 2011.
- [56] B. Gómez-Ferrer, R. Vila, D. Jiménez-Rey, C.J. Ortiz, F. Mota, J.M. García, A. Rodríguez, J. Nucl. Mater. 447 (2014) 225–232.
- [57] S. Takaki, J. Fuss, H. Kugler, U. Dedek, H. Schultz, Radiat. Eff. 79 (1983) 87–122.
- [58] H. Matsui, S. Takehana, M.W. Guinan, J. Nucl. Mater. 155–157 (1988) 1284–1289.
- [59] J.D. Torre, C.-C. Fu, F. Willaime, A. Barbu, J.-L. Bocquet, J. Nucl. Mater. 352 (2006) 42–49.
- [60] D.T. Gillespie, J. Chem. Phys. 115 (2001) 1716.
- [61] S.L. Grand, Broad-phase collision detection with CUDA, in: N. Corporation (Ed.), *GPU Gems 3*, 2008.
- [62] S. Chandrasekhar, Rev. Mod. Phys. 15 (1943) 1–89.
- [63] A.L. Nikolaev, T.E. Kurennykh, J. Nucl. Mater. 414 (2011) 374–381.
- [64] H. Abe, E. Kuramoto, J. Nucl. Mater. 283–287 (2000) 174–178.
- [65] A.L. Nikolaev, J. Phys.: Condens. Matter 11 (1999) 8633–8644.